

INSTITUT DE FORMATION AUX
NORMES ET TECHNOLOGIES
DE L'INFORMATIQUE
SOKODE

ReactJS

ReactJS

KONDI Abdoul Malik

IFNTI

12 janvier 2026

Table des matières

1. Introduction et fondamentaux
   - Frameworks
   - Pourquoi choisir React ?
2. Créer une première app React
3. JSX et Composant
   - Composants React
4. Créer un projet React.js
5. Structure d'un projet React créé avec Vite
6. Les Hooks
   - useState : Premier Hook React

Introduction et fondamentaux

Introduction à React.js



Définition

React.js est une bibliothèque JavaScript basée composants (composants **JSX**) permettant de construire des **Single Page Applications** (**SPA**).

Qu'est-ce qu'un framework front-end ?

Définition

Ensemble de classes, fonctions et utilitaires qui simplifient le développement d'applications web compatibles avec tous les navigateurs. Les frameworks front-end sont omniprésents dans les grandes entreprises : Facebook, Twitter, Amazon, ainsi que dans la majorité des startups.

Objectif

Ils facilitent la création d'applications web modernes et performantes. Éviter de réinventer la roue pour :

- gestion de l'interface utilisateur
- événements
- ...

Comparaison library / framework

Framework

Solution complète avec une architecture imposée.

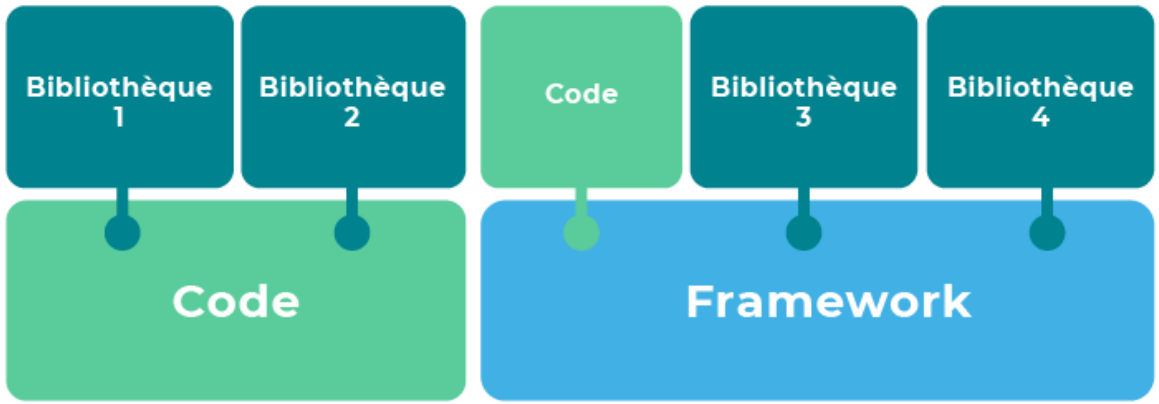
- Structure imposée.
- Outils intégrés : routing, API, etc.
- Exemple : Angular.

Library

Un ensemble d'outils intégrés à votre code, flexible.

- Flexible et composable.
- Ajoutée à un projet existant.
- Exemple : React.

React : library ou framework ?



Cas de React

Techniquement **React** est une bibliothèque, mais son écosystème est si vaste qu'il se comporte comme un framework.

Pourquoi choisir React ?

Une communauté massive

- Problèmes déjà rencontrés et documentés.
- Réponses sur GitHub, Stack Overflow.

Tendance du marché

- L'un des frameworks les plus utilisés.
- Très demandé dans les entreprises et startups.

Atouts de React

- Populaire
- Mature et fiable
- Support communautaire massif
- Opportunités professionnelles importantes

Créer une première app React

Transformer un document HTML en app React

App Reactjs

Pour réaliser notre toute première applicaiton React.js et comprendre les bases de React sans outils complexes (Webpack, bundlers), nous allons passer par Utilisation de liens **CDN** pour importer React dans un fichier **HTML**.

Qu'est-ce qu'un CDN ?

Un CDN (`Content Delivery Network`) permet de charger une bibliothèque directement via une balise `<script>`. Dans notre cas elle va nous permettre d'utiliser React immédiatement, sans installation préalable.

CDN React.js

Allez sur la doc de React.js

`https://fr.legacy.reactjs.org/docs/cdn-links.html`

Usage basique de React.js

Prérequis

Créer un fichier **index.html** contenant un document html de base. Ce document doit contenir un element (une `div`) avec l'id **root**.

script basique

```
<script>
    const rootTag = ReactDOM.createRoot(
        document.getElementById('root'));
    rootTag.render(React.createElement("button",
    null,"Salut"));
</script>
```

Usage basique de React.js

Explication du code source

- On crée un element racine `virtuelle` avec la fonction createRoot.

```
createElement(type, props, ...children)
```

`https://react.dev/reference/react/createElement#createelement`

- On Rajoute un nouvelle element React.js a notre élement racine.

Exercice : créer un menu de navigation

Objectif

Créer un menu de navigation que vous allez rajouter a l'élément racine en utilisant uniquement `React.createElement`.

Structure du menu

```
<ul>
   <li>Accueil</li>
   <li>Services</li>
   <li>Contact</li>
</ul>
```

JSX et Composant

Qu'est-ce que JSX ?

Définition

JSX (JavaScript `XML`) est une syntaxe proche du HTML utilisée dans React pour décrire l'interface utilisateur. Elle perment d'écrire des **balises** comme en **HTML** directement dans du **JavaScript** converties ensuite en appels `createElement`.

JSX vs createElement

Version JSX

```
<button>Salut</button>
```

Version createElement

```
React.createElement("button", null, "Salut");
```

Important

JSX n'est pas compris nativement par le navigateur. Il doit être transformé par un bundler comme **Babel**, **Vite**, ...

Règles importantes du JSX

Particularités syntaxiques

- Toutes les balises doivent être fermées :
  `<img src="" alt="" />`
- Les attributs s'écrivent en camelCase : `<input autoFocus/>`
- `class` devient `className`
- `for` devient `htmlFor`
- `style` reçoit un objet JS :
  `<div style={{width:50, backgroundColor:'blue'}}/>`

Outil

Des convertisseurs HTML → JSX existent pour faciliter l'écriture.

Interpolation dans JSX

Principe

On insère des variables dans la structure JSX grâce aux **accolades**.

Exemple

```
const text = 'Bonjour IFNTI L3'
const id = 'monid'

const myJsx = <h1 id={id}>{text}</h1>;
rootTag.render(myJsx);
```

Un seul élément racine

Contraintes

L'élément racine virtuel (créer a partir du ReactDOM : ReactDOM.createRoot) ne peut rentre qu'un **seul** et unique nœud JSX racine. Pour rendre plusieurs élément racine on utilise les **Fragment** qui sont des balises JSX : `<Fragment></Fragment>` ou `<></>`

Solution : Fragment

```
const baseNode = <>
        <p>Premier paragraphe</p>
        <p>Second paragraphe</p>
    </>
```

Enfants JSX valides et conditions

Enfants valides

- chaîne de caractères
- `null`, `undefined`, `false`
- tableau d'éléments JSX
- Élément JSX. (div, h1, ...)

Condition simple

```
<>
    {title && <h1>{title}</h1>}
    <p>Mon texte</p>
    {[<li>Link 1</li>, <li>Link 2</li>]}
</>
```

Listes et attribut key

Rappel

Lors d'un rendu par tableau (souvent via `map()`), chaque élément doit avoir une `key` unique.

Exemple

```
const todos = ['Task 1', 'Task 2', 'Task 3']
const listTaskJsx = <>
        <ul>
            {todos.map((todo, index) => (
                <li key={index}>{todo}</li>
            ))}
        </ul>
    </>
```

Qu'est-ce qu'un composant ?



Qu'est-ce qu'un composant ?

Définition

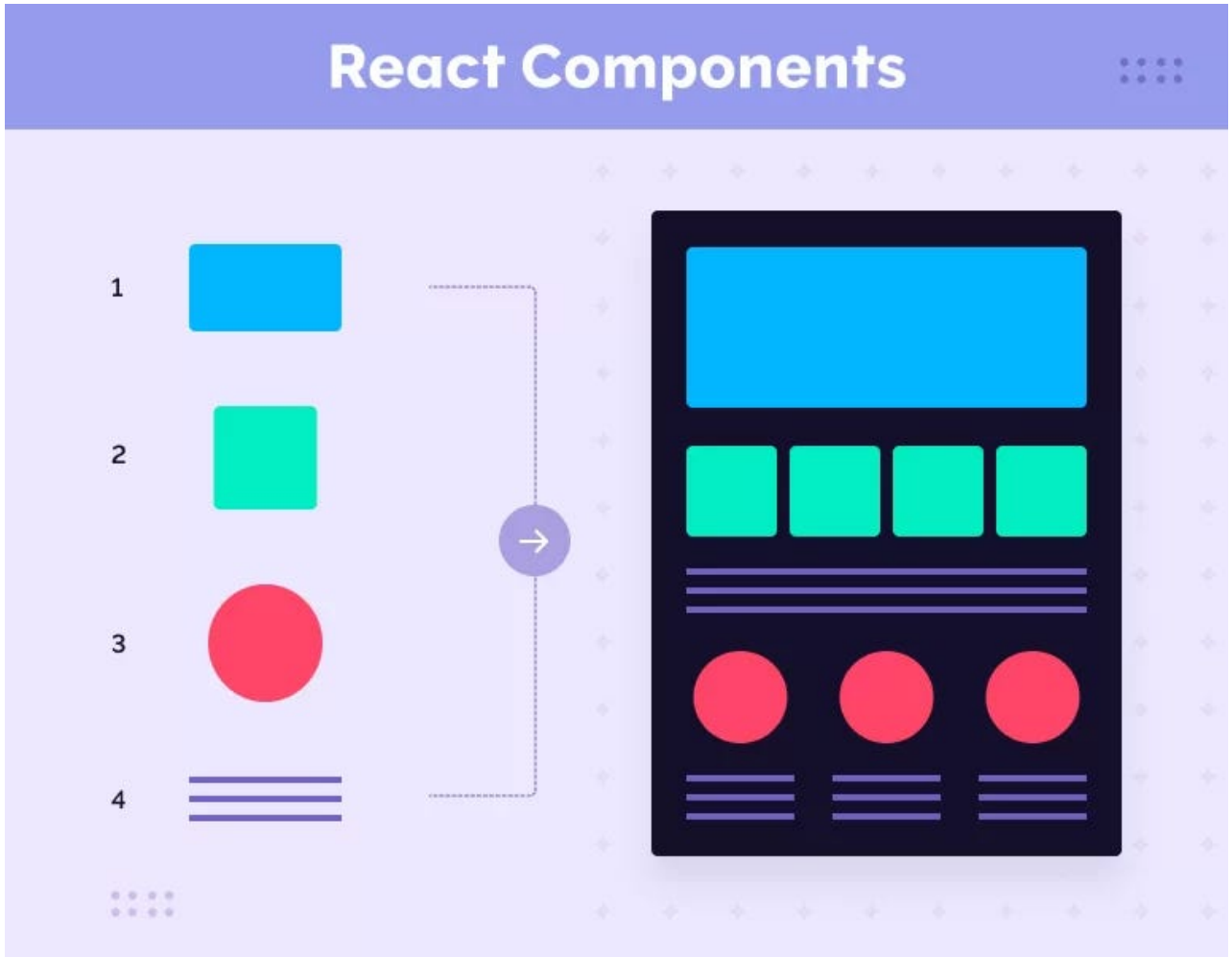
Un **composant React** est une fonction ou une classe **js** qui représente une partie indépendante de l'interface utilisateur. Il :

- doit retourne un élément **JSX**
- peut être réutilisé
- peut recevoir des données (props)

Attention

Une fonction **composant** doit renvoyer un **seul** nœud JSX racine.

Qu'est-ce qu'un composant ?



Structure d'un composant React

Éléments d'un composant fonctionnel

- **Code interne** : variables, constantes, logique métier.
- **Hooks (optionnel)** : état (`useState`), effets, etc.
- **Fonctions internes** : gestionnaires d'événements, helpers.
- **JSX** : structure visuelle retournée par le composant.

Créer un composant fonctionnel

Syntaxe moderne : fonction

```
function Welcome() {
  return <h1>Bonjour React !</h1>;
}
```

Utilisation

```
rootTag.render(<Welcome />);
```

Composants avec props

Définition

Les **props** sont des données envoyées à un composant.

Exemple

```
function Welcome(props) {
  return <h1>Bonjour {props.name} !</h1>;
}

rootTag.render(<Welcome name="Alice" />);
```

Créer un composant de classe

Définition

Un composant de classe est une ancienne manière de créer des composants React. Il repose sur l'héritage de `React.Component` et doit définir une méthode `render()`.

Syntaxe

```
class Welcome extends React.Component {
  render() {
    return <h1>Bonjour {this.props.name}</h1>;
  }
}
```

Écouteurs d'événements en JSX

Principe

Les événements s'écrivent en camelCase et reçoivent une fonction en paramètre.

Exemple

```
export function Show() {
    const handleClick = () => {
        alert('IFNTI')
    }

    return <button onClick={handleClick}>
        show IFNTI
    </button>
}
```

Bonnes pratiques composants

Recommandations

- Un composant = une responsabilité.
- Noms de composants toujours en majuscule.
- Préférer les composants fonctionnels. (Recommandé)
- Découper dès que possible en composant.

Créer un projet React.js

Créer un nouveau projet React.js

Trois types d'installtion

Il existe trois (3) méthode pour installer React.js.

- Via un framework : Next.js, React Router.
- From Scratch avec vite.js
- En l'ajoutant a un projet existant.

Attention

Dans notre cas nous allons le faire **From Scratch avec vite.js**

Créer un nouveau projet React.js : From Scratch

Syntaxe

```
npm create vite@latest app_name
-- -- --template react
```

Structure d'un projet React créé avec Vite

Structure du projet généré par Vite

Arborescence

```
app_name/
|-- node_modules/
|-- public/
|-- src/
|   |-- App.jsx
|   |-- app.css
|   |-- index.css
|   |-- main.jsx
|   |-- assets/
|-- index.html
|-- package.json
|-- vite.config.js
```

Structure du projet généré par Vite

Explication

- **index.html** : point d'entrée minimaliste utilisé par Vite.
- **main.jsx** : montage de l'app React.
- **App.jsx** : premier composant racine.
- **vite.config.js** : configuration du bundler.

Structure du projet personnalisé"

Explication

Pour ce cours nous allons spprimer les fichiers de styles et les assests dans le répertoire **src** et le vidé pour y placer un composant de base.

```
app_name/
|-- public/
|-- src/
|   |-- App.jsx
|   |-- main.jsx
|-- index.html
|-- package.json
|-- vite.config.js
```

Contenu du fichier main.jsx

Exemple réel

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

But

Monter le composant `App` dans l'élément HTML ayant l'id **root**.

Contenu du fichier App.jsx

Exemple basique

```
export default function App() {
  return (
    <div>
      <h1>Hello React avec Vite</h1>
    </div>
  )
}
```

Les Hooks

useState

Définition

`useState` est un hook permettant de gérer l'état interne d'un composant. Il permet :

- de stocker une valeur
- de déclencher un nouveau rendu quand cette valeur change

Important

Un hook ne peut être utilisé que dans un composant fonctionnel.

Syntaxe du useState

Structure

```
const [valeur, setValeur] = useState(valeurInitiale)
```

Explication

- **valeur** : état courant
- **setValeur** : fonction pour mettre à jour l'état
- **valeurInitiale** : valeur au premier rendu

Exemple : compteur simple 1/2

Code complet

```
import { useState } from 'react'

export default function Counter() {
  const [count, setCount] = useState(0)
  const increment = () => {
    setCount(count + 1)
  }
  return (
    <div>
      <button onClick={increment}>
        compteur : {count}
      </button>
    </div>
  )
```

Exemple : compteur simple 2/2

Effet

Chaque clic met à jour `count` et relance le rendu du composant.

Règles d'utilisation des Hooks

- Toujours appeler les hooks au **début** du composant.
- Ne jamais les mettre dans :
  - des conditions
  - des boucles
  - des fonctions internes
- Le nombre d'appels aux hooks doit toujours rester le même entre les rendus.

Deux signatures pour setState 1/2

Signature 1 : valeur directe

```
setCount(count + 1)
```

Utilisation

Quand la nouvelle valeur **ne dépend pas** de l'ancienne.

- remplacer une valeur
- activer/désactiver un état
- réinitialiser

Limite

Ne fonctionne pas correctement si plusieurs mises à jour successives utilisent la même valeur.

Deux signatures pour setState 2/2

Signature 2 : fonction basée sur l'état précédent

```
setCount(prev => prev + 1)
```

Utilisation

Quand la nouvelle valeur **dépend de l'ancienne**.

- increment / decrement
- plusieurs mises à jour en rafale

Important

Chaque appel reçoit la valeur **mise à jour**, ce qui garantit un comportement correct.

État complexe

Principe

React ne modifie jamais l'état directement. Les états complexes (objets, tableaux) doivent être **copiés** avant modification.

Interdit

```
state.user.age = 22    // Mauvais
todos.push("Nouvelle tache")  // Mauvais
```

La mutation directe ne déclenche pas de nouveau rendu et produit des bugs.

Modifier un objet avec le spread operator

Mauvaise pratique

```
setUser({ name: "tamba", age: user.age + 1 })
```

Chaque champ doit être réécrit manuellement, source d'erreurs.

Bonne pratique : spread operator

```
setUser({...prev, age: prev.age + 1})
```

Bonne pratique

- `...prev` copie toutes les propriétés
- on modifie uniquement le champ nécessaire

Modifier une liste

Exemple : ajouter un item

```
setItems([...prev, "new item"])
```

Bonne pratique

- `...prev` copie la liste
- on ajoute un élément à la fin

Résumé

Résumé

- Les objets et listes doivent être copiés avant modification.
- Utiliser systématiquement :
  - **Spread operator** (...)
- Ne jamais muter l'état : `push()`, `pop()`, `splice()`, assignation directe.

Règle d'or

Toujours retourner une **nouvelle** version de l'état, jamais modifier l'ancienne.

Le flux de données 1/4

Flux normal (One-Way Data Flow)

- Les données circulent **du parent vers l'enfant** via les props
- Un composant parent transmet des données à ses composants enfants
- Les enfants ne peuvent pas modifier directement les props reçues
- Principe fondamental : *les props sont en lecture seule*

Le flux de données 2/4

Flux normal (One-Way Data Flow)

```
function Parent() {
  const [message, setMessage] = useState("Bonjour")
  return <Enfant texte={message} />;
}

function Enfant({ texte }) {
  return <p>{texte}</p>; // Reçoit la donnée
}
```

Le flux de données 3/4

Le Flux inversé (Reverse Data Flow)

- L'enfant communique avec le parent via des **callbacks**
- Le parent passe une fonction comme prop à l'enfant
- L'enfant appelle cette fonction pour envoyer des données au parent

Le flux de données 4/4

Le Flux inversé (Reverse Data Flow)

```
function Parent() {
  const [valeur, setValeur] = useState("");
  return <>
    <p>{valeur}</p>
    <Enfant custom_action={setValeur} />
  </>;
}

function Enfant({ custom_action }) {
  return <input onChange={(e) => custom_action(
  e.target.value)} />;
}
```

Les inputs contrôlés 1/2

Principe

React utilise des **composants contrôlés** pour gérer les champs de saisie :

- L'état React devient la "source unique de vérité"
- La valeur de l'input est toujours synchronisée avec l'état
- Chaque modification déclenche une mise à jour de l'état

Les inputs contrôlés 2/2

Example

```
function InputControle() {
  const [texte, setTexte] = useState("");
  return (
    <div>
      <input
        value={texte}
        onChange={(e) => setTexte(e.target.value)}
        placeholder="Saisir du texte"
      />
      <p>Vous avez saisi : {texte}</p>
    </div>
  );
}
```

Les formulaires 1/2

Gestion des formulaires

Pour récupérer les données d'un formulaire :

- Utiliser l'événement `onSubmit` sur la balise `<form>`
- Appeler `e.preventDefault()` pour éviter le rechargement de la page

Les formulaires 2/2

Example

```
function Formulaire() {
  const handleSubmit = (e) => {
    e.preventDefault(); // Empêche le rechargement
    console.log(new FormData(e.target)); // Traitem
  };
  return (
    <form onSubmit={handleSubmit}>
      <input name="nom" type="texte" />
      <input name="email" type="email" />
      <button type="submit">Envoyer</button>
    </form>
  );
}
```

Exercice : Créer un formulaire d'inscription avec validation

Consignes 1/2

1. Créer un formulaire avec les champs : nom, email, mot de passe
2. Ajouter un boutton submit dans le formulaire
3. Utiliser `useState` pour gérer l'état de chaque champ
4. Ajouter une validation :
   - Le nom doit contenir au moins 3 caractères
   - L'email doit être valide
   - Le mot de passe doit contenir au moins 6 caractères

Exercice : Créer un formulaire d'inscription avec validation

Consignes 2/2

1. Afficher les messages d'erreur sous chaque champ
2. Désactiver le bouton si le formulaire n'est pas valide
3. À la soumission, afficher les données dans la console
4. Créer un composant réutilisable `Input` qui gère sa propre validation